PHY407: Lab Report 1

Numerical Errors, Simple Numerical Integrals and Derivatives

Gabrielle Gutierrez Sandoval

github: https://github.com/gabby06-del/PHY407-all-projects

1010605604

Gabrielle Gutierrez, 1010605604

Question 1: Exploring numerical issues with standard deviation calculations

Background

As per the lab report background and instructions, two formulas for finding the sample standard deviation were used: Formula (2) and Formula (3).

$$x = \frac{1}{n}\sum_{i-1}^{n} x_i \ , \ \sigma \equiv \sqrt{\frac{1}{n-1}\sum_{i-1}^{n}(x_i - x)^2} \quad (2)$$

$$\sigma \equiv \sqrt{\frac{1}{n-1}\left(\sum_{i=1}^{n} x_i^2 - nx^2\right)} \quad (3)$$

Formula two is the standard two pass method and formula three is the one pass method (Note: x is = to "x bar" due to formatting issues). Both values coming from formula (2) and (3) were tested against np.std as the "true value", first on the speed of light dataset, then on two random datasets with different means. Then a one pass variant using mathematical shifting was used to reduce error that arises from cancelation error.

Printed output for question 1:

```
Standard deviation for cdata using formula 2 : 0.07901054781905067
Standard deviation for cdata using formula 3 : 0.07901054763832621
Standard deviation for cdata using numpy package: 0.07901054781905067
Relative error for formula 2: 0.0
Relative error for formula 3: -2.2873460336752e-09
Std error using formula 2 for a1: 1.0049393146635544
Std error using formula 3 for a1: 1.0049393146635544
Std error using formula 2 for a2: 0.9982065851300489
Std error using formula 3 for a2: 0.980040937981705
Relative error for low_mean_data for formula 2: 0.0
Relative error for low_mean_data for formula 3: 0.0
Relative error for high_mean_data for formula 2: 0.0
Relative error for high_mean_data for formula 3: -0.018198284221875007
Formula (3) Relative Error cdata array: -2.2873460336752e-09
One-Pass Relative Error with Shift implemented cdata array: -1.7564474859226715e-16
Formula (3) Relative Error Low Mean Data: 0.0
Shifted One-Pass Relative Error Low Mean Data: 0.0
Formula (3) Relative Error High Mean Data: -0.018198284221875007
Shifted One-Pass Relative Error High Mean Data: 0.0
```

Written answers:

a. Using the given text file (cdata.txt) I found that formula 2 matches np.std with a relative error near zero, meaning it matches it almost exactly. Formula 3 has a higher relative error

calculated to be -2.2873460336752e-09. This is an expected result as formula 3 is not only computation heavy but also subtracts two large numbers almost equal in magnitude leading to possible catastrophic cancelation. Formula 2 avoids this as it takes small points and subtracts from each point which avoids this cancellation behavior.

b. For the dataset centered at 0, both formulas agree with np.std to some degree, leaving the errors to the order of machine precision (32-64 bytes as stated in class) since we are not running into areas with large cancellation. For the set shifted away from 0, formula (2) still closely follows np.std unlike formula (3) which shows a much larger relative error that is increasing the farther the set sits from 0.

c. For datasets far from 0 but the same order of magnitude a one pass method could avoid this large cancellation phenomenon. We can do this by shifting the data used by some constant K before summing, then after the shift is corrected algebraically.

$$Var = \frac{(\sum_{i=1}^{n}(x_i - K)^2) - \frac{\left(\left(\sum_{i=1}^{n}(x_i - K)^2\right)\right)}{n}}{n-1} \quad (4)$$

This while looking complicated still only requires one pass through the data, but the shift (x-K) is small and well-scaled helping us avoid cancellation. Applying this to the shifted dataset can reduce it further than what the machines precision is once again matching formula (2) precision with only one pass.

Question 2: Exploring roundoff error

Background

The function P(u) = $(1-u)^8$ was evaluated in its factored form and in its algebraically expanded form Q(u):

$$Q(u) = 1 - 8u + 28u^2 - 56u^3 + 70u^4 - 56u^5 + 28u^6 - 8u^7 + u^8 \quad (5)$$

over 500 points near u=1 in the range 0.97<u<1.02 (both evenly spaced and randomly sampled using np.random and linspace), to visually compare their behavior. Separately

$$f(u) = \frac{u^8}{u^4 \cdot u^4} \quad (6)$$

,which is analytically identically 1, was evaluated over the same range to directly expose floating-point rounding error.

Output for question 2

a. The Q(u) plot is visibly noisier than the P(u) plot near u=1. This happens because Q(u) is computed as a sum of terms that are individually much larger in magnitude than the true result near u=1. Summing large terms that mostly cancel is a classic source of catastrophic round off error, amplified further by the many separate arithmetic operations involved.

Unlike Q(u), P(u) computes $(1-u)^8$ directly as a single small quantity raised to a power, with no large-term cancellation, so it stays smooth.

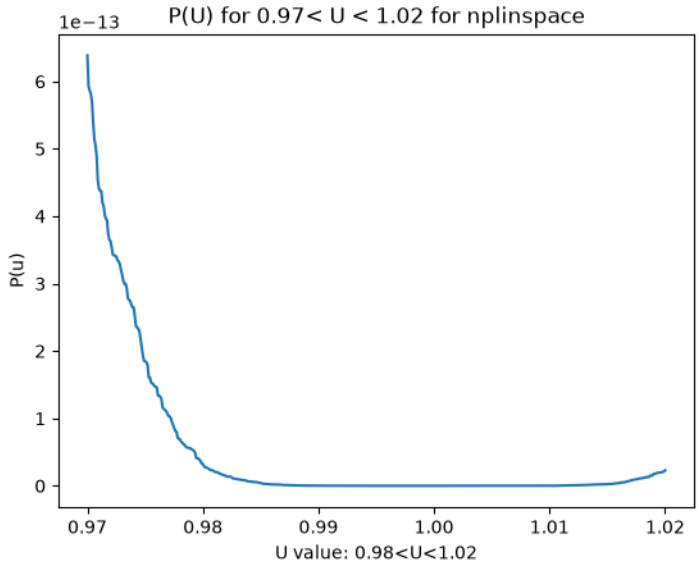


Figure 1: $P(u) = (1-u)^8$, evaluated in its factored form for 500 evenly spaced points $0.97 < u < 1.02$. [nprnad]

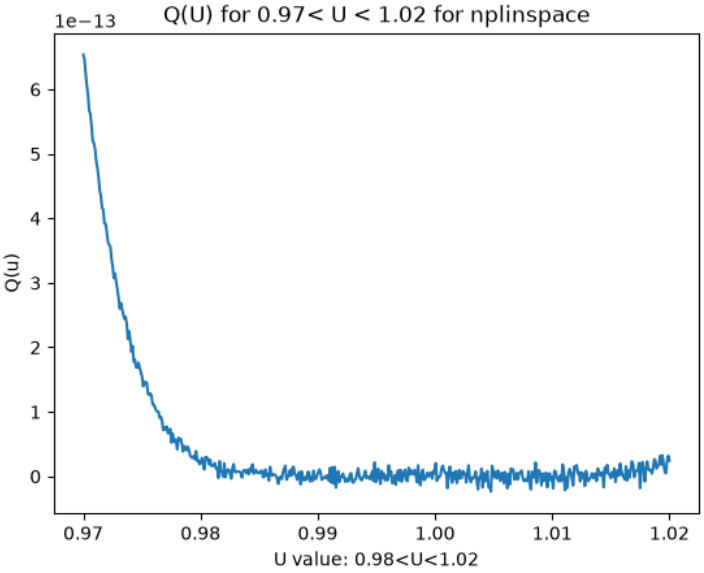


Figure 2: $Q(u) = (1-u)^8$, evaluated in its expanded form (formula 6) from for 500 evenly spaced points $0.97 < u < 1.02$. [np.rand]

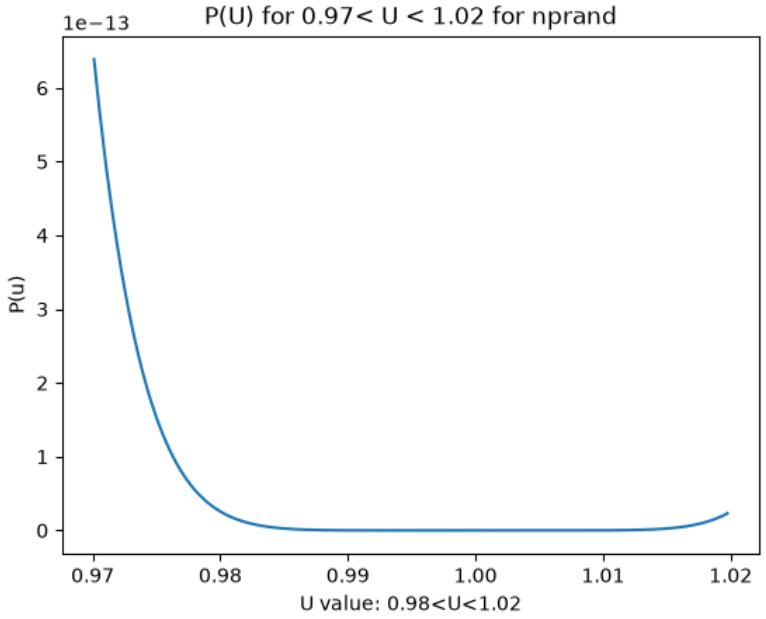


Figure 3: $P(u) = (1-u)^8$ evaluated in its factored form for 500 evenly spaced points $0.97 < u < 1.02$. [nplinspace]]

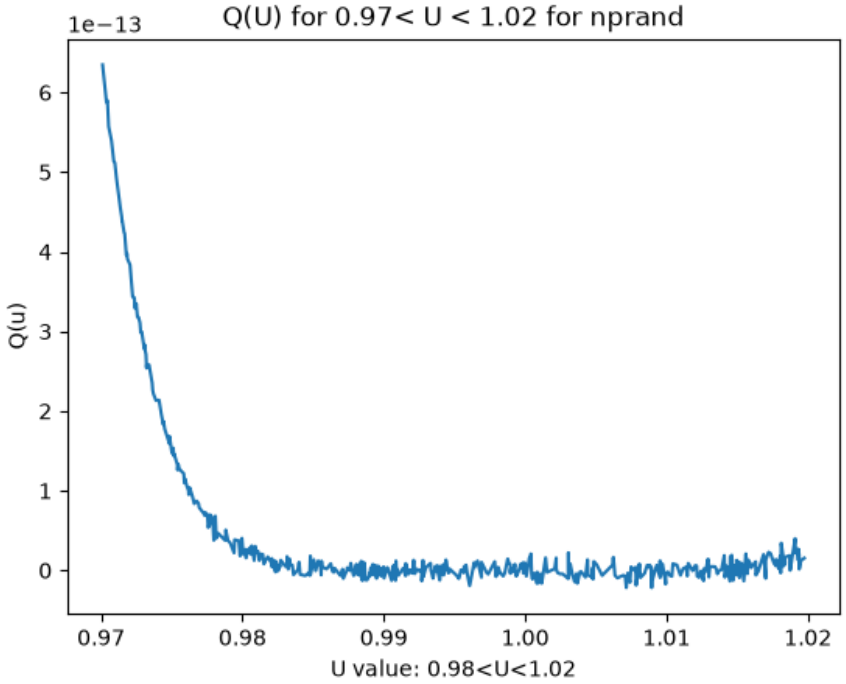


Figure 4: $Q(u) = (1-u)^8$, evaluated in its expanded form (formula 6) from for 500 evenly spaced points $0.97 < u < 1.02$. [np.linspace]

b. Numerical output for std dev for f(set_linspace): 1.3248454680138175e-16
f(u) should equal exactly 1 for all u analytically but realistically fluctuates around 1 due to floating point rounding throughout the repeated calculation. Plotting f-1 shows this rounding error. This matches Equation (1)'s prediction that the error introduced by each floating-point operation is bounded by roughly times the magnitude of the numbers involved; since u≈1, the operations don't amplify the error, so the standard deviation of f(u) over this range confirms the round off origin of the fluctuations.

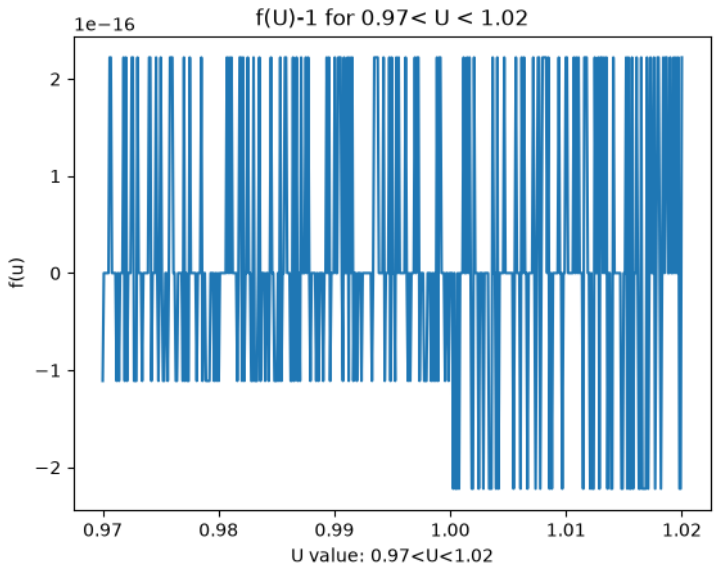


Figure 5: f(u) - 1, where f(u) (6) is analytically identical to 1, showing pure round-off error.

Question 3: Trapezoidal and Simpson's Rule for Integration

Background:

For this question we evaluated the integral,

$$\int_0^1 \frac{4}{1+x^2}\,dx \text{ (a)}$$

In various different ways. Initially (using Wolfram Alpha) the exact value of the integral was found to be π. To evaluate this integral using computational methods, the lab guidelines instructed us to use Trapezoidal rule (7) and the Simpsons rule (8).

$$I_{trap} = \frac{h}{2}[f(x_0) + 2\sum_{i=1}^{n-1} f(x_i) + f(x_n)] \text{ (7)}$$

$$I_{simp} = \frac{h}{3}\left[f(x_0) + 4\sum_{i=1,\,odd}^{n-1} f(x_i) + 2\sum_{i=2,\,even}^{n-2} f(x_i) + f(x_n)\right] \quad (8)$$

I first used this rule with a fixed n=4 then increasing N by powers of 2 until each method's absolute error dropped below 10^{-9} (recording the required N – slices and computation time needed to reach the needed number of slices), and finally using Richardson-style error estimation (9) with the Trapezoidal Rule at N_1=16 and N_2=32 (and associated I_1 and I_2).

$$\varepsilon_2 = \frac{(I_2 - I_1)}{3} \quad (9)$$

Printed output

```
Initial calculation with N = 4:
trapezoidal rule:  3.1311764705882354
simpsons rule:     3.14156862745098
Target reached at n for trapezoidal rule= 14 N = 16384
Value calculated for trapezoidal rule: 3.1415926529689115
Absolute error for trapezoidal rule: 6.208815683805824e-10
Computation time for trapezoidal rule: 0.005000000783184078 ms

Target reached at n for Simpson's rule= 5 N = 32
Value calculated for simpson's rule: 3.1415926535528365
Absolute error for simpson's rule: 3.695665995451236e-11
Computation time for simpson's rule: 0.0027090000003227033 ms

I1 trapezoidal (N1 = 16): 3.140941612041389
I2 trapezoidal (N2 = 32): 3.1414298931749745
Estimated error ϵ2:  0.00016276037786185546
```

Question 3 written answers

a. The exact value of the integral $\int_0^1 \frac{4}{1+x^2}dx = \pi \approx 3.14159$ (computed using Wolfram alpha)

b. For N=4 slices the Simpson's rule, and the Trapezoidal rule came very close to the exact value computed by wolfram alpha (π). The trapezoidal rule resulted in 3.1311764705882354, and the Simpson's rule resulted in 3.14156862745098. They were both accurate to the first decimal place however the Simpson's rule showed greater accuracy to the exact value than the Trapezoidal rule even with a low N value. This is expected as Simpson's rule has $O(h^4)$ truncation error compared to the trapezoidal rule which has $O(h^2)$. (truncation error is the difference between the approximate solution y_n and the exact solution $y(x_n)$ at step n.[1])

c. For this question we needed to find the number of N slices needed to approximate the integral with an error of $O(10^{-9})$. Instead of brute forcing it, I created a loop that would break when the error for each rule was less than or equal to the target error. This was achieved for the Trapezoidal rule at N= 16384 and for Simpson's rule at N= 32. For the

loop to find the exact N it took it 0.00504200579598546 ms for the trapezoidal rule and 0.002707995008677244 ms for Simpson's rule, cutting the computation time almost by half.

d. Using equation (9) (as seen above) for the Trapezoidal method the error estimate was found to be 0.00016276037786185546.
e. This method employed in Q3(d) would not work for Simpson's rule as it was derived directly from the Trapezoidal rule, specifically from its leading order truncation error term $O(h^2)$. This means that the coefficient only allows the I_2 and I_1 terms' difference to cancel via the factor of three which is seen in equation (9) (this is due to 2^2-1=3). Simpson's Rule has a different convergence order of $O(h^4)$, so the same derivation does not apply, but the same idea can still be applied. Redoing the derivation using Simpson's Rule's own leading order error term would instead produce a different scaling coefficient leading the expression to be divided by 15 (from 2^4-1=15) rather than 3 in order to fully isolate Simpson's leading-order error term when combining estimates at N and 2N.

Question 4: Simple Numerical Differentiation

Background:

For this question we were asked to consider the function,

$$f(x) = e^{-x^2} \quad (10)$$

We were then tasked with finding the analytical derivative of our function at $x = \frac{1}{2}$, first I used the chain rule to find the derivative, then compared this value against forward-difference and central-difference numerical approximations, over 17 values of h from 10^{-16} to 10^{0} (increasing by a factor of 10 each step). The absolute relative error of each numerical estimate was computed and plotted on a log-log scale for both methods (utilized plt.loglog)

Printed output

```
Analytical (exact) value of f'(0.5): -0.7788007830714049
h = 1.0e-16    f_num = -1.1102230246    rel_err = 4.2555458181e-01
h = 1.0e-15    f_num = -0.7771561172    rel_err = 2.1117927326e-03
h = 1.0e-14    f_num = -0.7771561172    rel_err = 2.1117927326e-03
h = 1.0e-13    f_num = -0.7793765633    rel_err = 7.3931643107e-04
h = 1.0e-12    f_num = -0.7788214518    rel_err = 2.6539140165e-05
h = 1.0e-11    f_num = -0.7787992473    rel_err = 1.9719514714e-06
h = 1.0e-10    f_num = -0.7788014678    rel_err = 8.7915769227e-07
h = 1.0e-09    f_num = -0.7788008016    rel_err = 2.3824943182e-08
h = 1.0e-08    f_num = -0.7788007905    rel_err = 9.5693973638e-09
h = 1.0e-07    f_num = -0.7788008216    rel_err = 4.9484925655e-08
h = 1.0e-06    f_num = -0.7788011724    rel_err = 4.9996017351e-07
h = 1.0e-05    f_num = -0.7788046770    rel_err = 4.9999085329e-06
h = 1.0e-04    f_num = -0.7788397166    rel_err = 4.9991667049e-05
h = 1.0e-03    f_num = -0.7791895344    rel_err = 4.9916662537e-04
h = 1.0e-02    f_num = -0.7826298571    rel_err = 4.9166284123e-03
```

```
h = 1.0e-01     f_num = -0.8112445700    rel_err = 4.1658647035e-02
h = 1.0e+00     f_num = -0.6734015585    rel_err = 1.3533528324e-01
Minimum error occurs at h = 1.0e-08, with error = 9.569397364e-09
h = 1.0e-16     f_num = -1.1102230246    rel_err = 4.2555458181e-01
h = 1.0e-15     f_num = -0.7771561172    rel_err = 2.1117927326e-03
h = 1.0e-14     f_num = -0.7771561172    rel_err = 2.1117927326e-03
h = 1.0e-13     f_num = -0.7788214518    rel_err = 2.6539140165e-05
h = 1.0e-12     f_num = -0.7788214518    rel_err = 2.6539140165e-05
h = 1.0e-11     f_num = -0.7787992473    rel_err = 1.9719514714e-06
h = 1.0e-10     f_num = -0.7788009126    rel_err = 1.6638040136e-07
h = 1.0e-09     f_num = -0.7788008016    rel_err = 2.3824943182e-08
h = 1.0e-08     f_num = -0.7788007850    rel_err = 2.4416244548e-09
h = 1.0e-07     f_num = -0.7788007828    rel_err = 4.0948470885e-10
h = 1.0e-06     f_num = -0.7788007830    rel_err = 5.3096063399e-11
h = 1.0e-05     f_num = -0.7788007830    rel_err = 8.8735070499e-11
h = 1.0e-04     f_num = -0.7788007766    rel_err = 8.3334298518e-09
h = 1.0e-03     f_num = -0.7788001341    rel_err = 8.3333301471e-07
h = 1.0e-02     f_num = -0.7787358857    rel_err = 8.3329916757e-05
h = 1.0e-01     f_num = -0.7723373145    rel_err = 8.2992579566e-03
h = 1.0e+00     f_num = -0.3367007793    rel_err = 5.6766764162e-01
Minimum error occurs at h = 1.0e-06, with error = 5.3096063399e-11
```

Figure output (Q4(c))

(c)

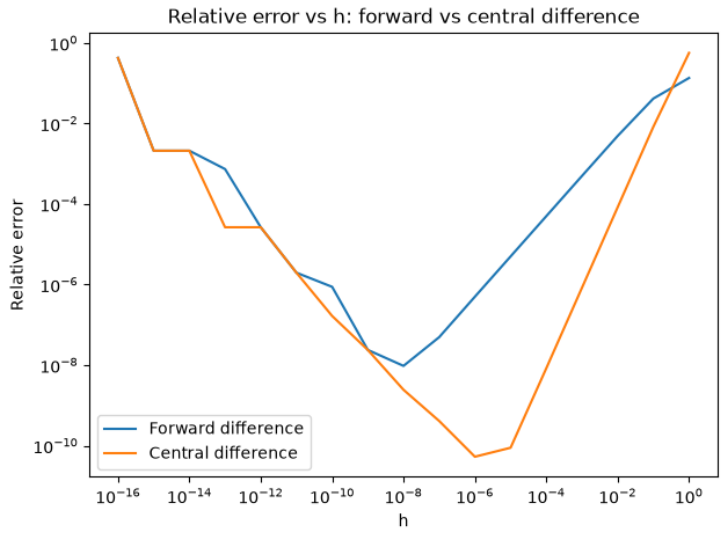


Figure 6: log-log plot to compare the relative error vs h for forward difference and central difference calculations.

Written answers:

a. The forward-difference minimum error occurs at h ≈ 10^{-8}. This matches theory as forward difference has O(h) truncation error which is consistently shrinking with h but O(ε/h) round-off error has consistent growth as h shrinks. Balancing these values that are increasing and decreasing as a function of h predicts an optimal h on the order of $\sqrt{\varepsilon_{machine}} = 10^{-8}$, exactly matching the observed minimum as listed above.
b. The central-difference minimum error occurs at h ≈ 10^{-6}. This matches the theoretical result as central difference has a O(h^2) truncation error, so balancing against O(ε/h) round-off predicts an optimal h = $\sqrt[3]{\varepsilon_{machine}} \approx 10^{-5}$, which was close to the observed h ≈ 10^{-6}. Because truncation error falls off faster (h^2 vs h), central difference can handle a larger h and reaches an overall much lower minimum error.
c. The plot (as seen above under figure output) shows that both central difference and forward difference have parabolic shapes and follow the same overall trend however it can be seen that central difference has a lower overall relative error compared to forward difference for the same step size. However, this behavior does not happen at every point on the graph, at very low h values they converge on the same error values, making them have the same relative error. This may be less due to the accuracy of the difference scheme and more due to roundoff error and machine precision limitations.
d. Following the rationale from Q4(c) at high h values approximation (or as I've been referring to it throughout this lab, truncation) error dominates. This is due to the Taylor approximation truncation which grows with growing h values (or in the central difference case h^2). At low h values round-off error dominates as the values get increasingly small trending towards zero; their values become essentially identical only differing by one or two numbers. At a sufficiently low h, machine precision limitations will show, and the error value will appear the same due to the marginal differences between them not being able to be captured by the computer leading to catastrophic cancellation. This rise and fall of each respective error is what provides the plot with its parabolic shape for each difference scheme; as the h value gets larger, the round-off error drops and the truncation error rises this is also why the point at the "vertex" of the parabola is seen the be the point with the lowest error and highest accuracy.

AI disclosure:

AI used: Claude AI

Prompt: "Are there any bugs in my code or anything that is unclear or logically flawed? Also is there any equation that I wrote that is wrong?"

- I showed the AI my code for Q2, Q3, and Q4 to ensure that all my equations were correct

AI answer: I used this reply to remove any redundancies and clean up my code

Gabrielle Gutierrez, 1010605604

I went through all three files. Nothing is going to crash or give you wrong numbers in the ways that matter for your results, but there are a few real issues worth fixing.

Lab1Q2

1. Dead/duplicate code in P() and f()

python
```
def P(u_set):
    P= (1-u_set)**8      # this line does nothing — computed and thrown away
    return (1-u_set)**8   # recomputes the same thing
```

Same pattern in f():

python
```
def f(u_set):
    f= (u_set**8) /((u_set**4)*(u_set**4))   # unused
    return (u_set**8 )/((u_set**4)*(u_set**4))
```

Not wrong, just wasted computation and confusing (the local variable name shadows the function name, which can trip you up if you ever try to reference P or f inside the function later). Just do return (1-u_set)**8 and drop the first line.

2. Mislabeled x-axis
Every plot's xlabel says "U value: 0.98<U<1.02", but your actual arrays (set1, set2) span 0.97 to 1.02, and your plot titles correctly say 0.97. This looks like a leftover typo — the axis label doesn't match the data or the title.

3. Equation check — Q(u) polynomial
Your expansion of $(1-u)^8$:

$$1 - 8u + 28u^2 - 56u^3 + 70u^4 - 56u^5 + 28u^6 - 8u^7 + u^8$$

This is correct — matches row 8 of Pascal's triangle (1,8,28,56,70,56,28,8,1) with alternating signs. ☑

4. f(u) = u^8/(u^4·u^4)
Algebraically this is identically 1 everywhere — that's the point of the exercise (showing that a numerically "trivial" identity picks up roundoff error near u=1), so this is intentional, not a bug.

Lab1Q3

I checked the trapezoidal, Simpson's, and Richardson error-estimate formulas line by line:

- Trapezoidal: $\frac{h}{2}\left(y_0 + 2\sum y_{mid} + y_N\right)$ ☑
- Simpson's: $\frac{h}{3}\left(y_0 + 4\sum y_{odd} + 2\sum y_{even} + y_N\right)$ ☑
  (your odd/even index slicing y[1:N:2] and y[2:N:2] is correct)

Gabrielle Gutierrez, 1010605604

- Richardson extrapolation: $\epsilon_2 = (I_2 - I_1)/3$ ☑ — this is the correct error estimate for trapezoidal rule when doubling N.

No logic or equation errors here. One thing to be aware of (not a bug): your for n in range(2, 30) loop could in principle run up to $N = 2^{29} \approx 537$ million points if the target error is never hit — fine here since Simpson's converges fast, just worth knowing why it's structured that way.

Lab1Q4

Derivative and difference formulas are correct:

- Forward: $\frac{f(x+h)-f(x)}{h}$ ☑
- Central: $\frac{f(x+h)-f(x-h)}{2h}$ ☑
- Exact derivative of e^{-x^2} is $-2xe^{-x^2}$ ☑

Two small things, not conceptual errors:

1. Missing space in your print statements

python
```
print(f"\nMinimum error occurs at h = {h_values[min_i_for]}," f"with error = {rel_err_for[min_i_for]}")
```

These are two f-strings concatenated with no space between them, so the output reads h = 1e-08,with error = ... (no space after the comma). Add a space before "with".

2. h_values = 10.0 ** np.arange(-16.0, 1.0)
Computing powers of 10 this way can introduce tiny floating-point rounding (e.g., you might get 9.999999999999999e-09instead of exactly 1e-08). It won't affect your conclusions about optimal h, but if you want exact round values for a table, np.logspace(-16, 0, 17) is the more standard/precise way to generate this.

Bottom line: your math/equations are all correct in all three files — the only actual bugs are the mismatched xlabel in Q2 and the missing space in Q4's print formatting; everything else flagged is cosmetic/style.